

APIOBike

APIOBike APIO şəhərində yeni istifadəyə verilmiş velosiped paylaşım xidmətidir. Şəhər boyu bir neçə stansiya qurulub və bu, istifadəçilərə istənilən stansiyadan velosiped götürməyə və ya qaytarmağa imkan verir. Rahatlığı və aşağı qiyməti sayəsində o, qısa müddətdə APIO şəhərində ən vacib nəqliyyat növünə çevrilmişdir. Lakin, APIOBike bütün velosiped paylaşım xidmətlərinin qarşılaşdığı ümumi bir problemlə üzləşib: müxtəlif stansiyalarda velosipedlərin götürülməsi və qaytarılması dərəcələri arasındakı balanssızlıq. Bu, bəzi stansiyalarda çox az velosipedin qalmasına və yaxınlıqdakı sakinlərin velosiped götürə bilməməsinə, digər stansiyaların isə yerli sakinlərin ehtiyac duymadığı velosipedlərlə həddindən artıq dolmasına səbəb olur.

Bu problemi həll etmək üçün, APIOBike hər gecə velosipedləri stansiyalar arasında yenidən bölüşdürmək və hər bir stansiyada müvafiq sayda velosipedin olmasını təmin etmək məqsədilə balanslaşdırma yük maşını göndərməyi planlaşdırır. APIO şəhərində cəmi N stansiya var və onlar $0, 1, \dots, N - 1$ kimi nömrələnmişdir. Müşahidələr nəticəsində APIOBike müəyyən edib ki, hər axşam i stansiyasında velosipedlərin sayı həmişə sabit $A[i]$ dəyərində olur və gecələr heç kim velosiped götürmür və ya qaytarmır. Şirkət hər səhər i stansiyasında tam olaraq $B[i]$ sayda velosipedin olmasını hədəfləyir.

Bu N stansiya arasında balanslaşdırma yük maşını üçün ayrılmış $N - 1$ xüsusi yol var. Hər bir yol iki fərqli stansiyanı birləşdirir və yük maşını yalnız bu yollarda hərəkət edə bilər. Hər bir yolun uzunluğu bir vahiddir. Bu stansiyalar şəbəkəsi bir-birinə bağlıdır, yəni yük maşını istənilən stansiya cütü arasında hərəkət edə bilər. Hər gecə, APIOBike hər bir i stansiyasında tam olaraq $B[i]$ velosiped olmasını təmin etmək üçün balanslaşdırma maşını göndərməlidir. Yük maşını istənilən stansiyadan başlaya və marşrutunu istənilən stansiyada bitirə bilər.

Balanslaşdırma başlayarkən yük maşını boş olur. Yük maşını hər hansı bir stansiyada olduqda, sürücü stansiyadan istənilən sayda velosipedi maşına yükləyə, və ya maşından istənilən sayda velosipedi stansiyaya boşalda bilər. Yük maşınının və stansiyaların velosiped tutumu limitsizdir, lakin hər hansı bir yerdəki velosipedlərin sayı heç vaxt sıfırdan aşağı düşməməlidir. Şirkət balanslaşdırmanı tamamlamaq üçün mümkün olan minimum səyahət məsafəsini və ona uyğun strategiyanı müəyyən etmək istəyir.

İcra Detalları

Siz aşağıdakı proseduru icra etməlisiniz.

```
std::pair<std::vector<int>, std::vector<long long>>
find_rebalancing_strategy(int N,
                           std::vector<int> A,
                           std::vector<int> B,
                           std::vector<int> U,
                           std::vector<int> V)
```

- A : N uzunluğunda massiv, burada $A[i]$ hər axşam i stansiyasındakı velosipedlərin sayıdır.
- B : N uzunluğunda massiv, burada $B[i]$ hər səhər i stansiyasında olması hədəflənən velosipedlərin sayıdır.
- U, V : yol şəbəkəsini təmsil edən $N - 1$ uzunluğunda massivlər. Hər bir $0 \leq i < N - 1$ üçün, i -ci yol $U[i]$ və $V[i]$ stansiyalarını birləşdirir.
- Bu prosedur hər test nümunəsi üçün **ən çox 150 000 dəfə** çağırılır.

Prosedur balanslaşdırma strategiyasını təmsil edən $k + 1$ uzunluqlu (X, Y) massivlər cütünü qaytarmalıdır:

- X : xronoloji ardıcılıqla ziyarət edilən stansiya indekslərinin ardıcılığı.
- Y : ziyarət edilən hər stansiyada xalis velosiped təhvilini göstərir. Hər bir $0 \leq j \leq k$ üçün:
 - Əgər $Y[j] \geq 0$ olarsa, sürücü $X[j]$ stansiyasında maşından $Y[j]$ velosiped boşaldır və stansiyanın velosiped sayını $Y[j]$ qədər artırır.
 - Əgər $Y[j] < 0$ olarsa, sürücü $X[j]$ stansiyasından maşına $-Y[j]$ velosiped yükləyir və stansiyanın velosiped sayını $-Y[j]$ qədər azaldır.

Səyahət məsafəsi k olan **keçərli** (X, Y) balanslaşdırma strategiyası aşağıdakı şərtləri ödəməlidir:

- Hər bir $0 \leq j \leq k$ üçün, $0 \leq X[j] < N$.
- Hər bir $0 \leq j < k$ üçün, $X[j]$ və $X[j + 1]$ stansiyaları yolla birbaşa birləşdirilmişdir.
- Hər bir $0 \leq j \leq k$ üçün, $\sum_{t=0}^j Y[t] \leq 0$.
- Hər bir $0 \leq i < N$ stansiyası və hər bir $0 \leq j \leq k$ addımı üçün, $S_{i,j}$ $0 \leq t \leq j$ və $X[t] = i$ şərtlərini ödəyən bütün t addımları üzrə $Y[t]$ -lərin cəmi olsun.
 - i stansiyasındakı velosipedlərin sayı heç vaxt sıfırdan aşağı düşmür: $A[i] + S_{i,j} \geq 0$.
 - Strategiya başa çatdıqdan sonra hər bir i stansiyasında tam olaraq $B[i]$ velosiped olmalıdır: $A[i] + S_{i,k} = B[i]$.

Məhdudiyyətlər

- $2 \leq N \leq 300\,000$
- Hər test nümunəsində `find_rebalancing_strategy` çağırışları üzrə N -lərin cəmi 300 000-
i keçmir.
- $0 \leq i < N$ şərtini ödəyən hər bir i üçün $0 \leq U[i], V[i] < N$ və $U[i] \neq V[i]$.

- $0 \leq i < N$ şərtini ödəyən hər bir i üçün $0 \leq A[i], B[i] \leq 10^9$.
- İstənilən stansiya cütü arasında hərəkət etmək mümkündür.
- $\sum_{i=0}^{N-1} A[i] = \sum_{i=0}^{N-1} B[i]$.
- $0 \leq i < N$ və $A[i] \neq B[i]$ şərtlərini ödəyən ən azı bir i mövcuddur.

Alt tapşırıqlar və Qiymətləndirmə

Burada T , `find_rebalancing_strategy` funksiyasının çağırılma sayıdır və $\sum N$ isə bütün `find_rebalancing_strategy` çağırışları üzrə N -lərin cəmidir.

Alt tapşırıq	Bal	Əlavə Məhdudiyyətlər
1	4	Stansiyalar və yollar P xassəsini ödəyir (aşağıya baxın). $0 \leq i < N$ və $A[i] > 0$ şərtlərini ödəyən tam olaraq bir i mövcuddur.
2	11	$T \leq 10$. $N \leq 7$. Stansiyalar və yollar P xassəsini ödəyir.
3	16	Stansiyalar və yollar P xassəsini ödəyir.
4	9	$0 \leq i < N$ və $A[i] > 0$ şərtlərini ödəyən tam olaraq bir i mövcuddur.
5	24	$\sum N \leq 500$.
6	15	$\sum N \leq 5000$.
7	21	Əlavə məhdudiyyət yoxdur.

P xassəsi: Hər bir $0 \leq i < N - 1$ üçün $U[i] = i$ və $V[i] = i + 1$. Hər bir $0 \leq i < N$ üçün $A[i] \neq B[i]$.

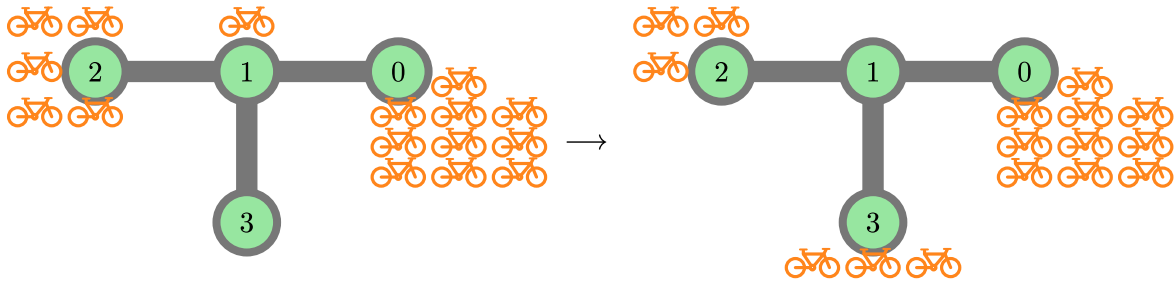
Qaytarılan X və Y massivlərinin uzunluğu tam olaraq $k^* + 1$ olarsa, burada k^* mümkün olan minimum səyahət məsafəsidir, lakin (X, Y) keçərli bir strategiya deyilsə, balın 50%-ni alacaqsınız.

Nümunələr

Nümunə 1

Aşağıdakı çağırışı nəzərdən keçirək:

```
find_rebalancing_strategy(4,
                        [10, 1, 5, 0],
                        [10, 0, 3, 3],
                        [0, 1, 1],
                        [1, 2, 3])
```



APIO şəhərində $N = 4$ stansiya var. Başlanğıcda stansiyalarda $A = [10, 1, 5, 0]$ velosiped var və məqsəd hər stansiyada $B = [10, 0, 3, 3]$ velosipedin olmasıdır.

Fərz edək ki, balanslaşdırma yük maşını 2 nömrəli stansiyadan başlayır və aşağıdakı addımları izləyir:

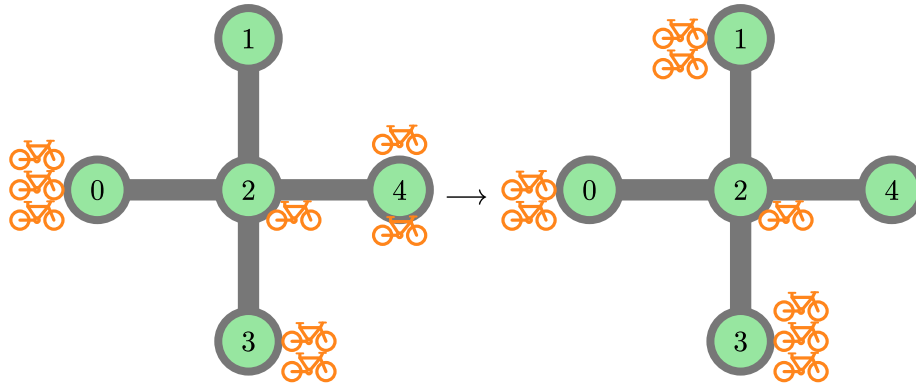
- 2 nömrəli stansiyada, maşına 2 velosiped yükləyin. İndi 2 stansiyasında 3 velosiped, maşında isə 2 velosiped var.
- 1 nömrəli stansiyaya keçin və maşına 1 velosiped yükləyin. İndi 1 stansiyasında 0 velosiped, maşında isə 3 velosiped var.
- 3 nömrəli stansiyaya keçin və 3 velosiped boşaldın. İndi 3 stansiyasında 3 velosiped, maşında isə 0 velosiped var.

Ümumi hərəkət məsafəsi 2-dir. Bu strategiya üçün uyğun olaraq qaytarılan massivlər $X = [2, 1, 3]$ və $Y = [-2, -1, 3]$ -dür. Sübut etmək olar ki, bu strategiya minimum məsafəni təmin edir. Beləliklə, prosedur $([2, 1, 3], [-2, -1, 3])$ qaytarmalıdır.

Nümunə 2

Aşağıdakı çağırışı nəzərdən keçirək:

```
find_rebalancing_strategy(5,
                        [3, 0, 1, 2, 2],
                        [2, 2, 1, 3, 0],
                        [2, 2, 2, 2],
                        [0, 4, 3, 1])
```



APIO şəhərində $N = 5$ stansiya var. Başlanğıcda stansiyalarda $A = [3, 0, 1, 2, 2]$ velosiped var və məqsəd hər stansiyada $B = [2, 2, 1, 3, 0]$ velosipedin olmasıdır.

Fərz edək ki, balanslaşdırma yük maşını 0 nömrəli stansiyadan başlayır və aşağıdakı addımları izləyir:

- 0 stansiyasında 1 velosiped yükləyin.
- 2 stansiyasına keçin və 1 velosiped yükləyin.
- 1 stansiyasına keçin və 2 velosiped boşaldın.
- 2 stansiyasına keçin.
- 4 stansiyasına keçin və 2 velosiped yükləyin.
- 2 stansiyasına keçin və 1 velosiped boşaldın.
- 3 stansiyasına keçin və 1 velosiped boşaldın.

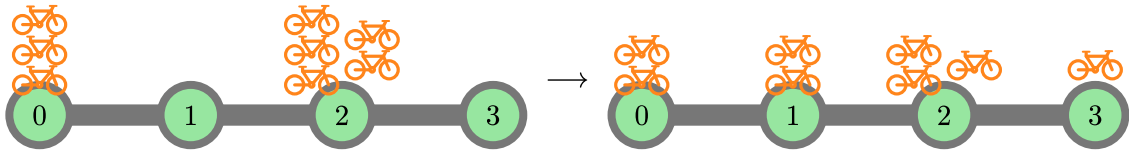
Ümumi hərəkət məsafəsi 6-dır. Bu strategiya üçün uyğun olaraq qaytarılan massivlər $X = [0, 2, 1, 2, 4, 2, 3]$ və $Y = [-1, -1, 2, 0, -2, 1, 1]$ -dir. Sübut etmək olar ki, bu strategiya minimum məsafəni təmin edir. Beləliklə, prosedur $([0, 2, 1, 2, 4, 2, 3], [-1, -1, 2, 0, -2, 1, 1])$ qaytarmalıdır.

Məsafəsi 6 olan digər keçərli strategiyalar, məsələn, $X = [0, 2, 3, 2, 4, 2, 1]$ və $Y = [-1, 0, 1, 0, -2, 0, 2]$ da düzgün hesab olunur. $7 = 6 + 1$ uzunluğunda massivlər qaytarılırsa, lakin keçərli strategiya deyilsə, məsələn $X = Y = [0, 0, 0, 0, 0, 0, 0]$ olarsa, xalın 50%-i veriləcəkdir.

Nümunə 3

Aşağıdakı çağırışı nəzərdən keçirək:

```
find_rebalancing_strategy(4,
                          [3, 0, 5, 0],
                          [2, 2, 3, 1],
                          [0, 1, 2],
                          [1, 2, 3])
```



Optimal həll yolu $X = [2, 1, 0, 1, 2, 3]$ və $Y = [-1, 1, -1, 1, -1, 1]$ -dir. Prosedur $([2, 1, 0, 1, 2, 3], [-1, 1, -1, 1, -1, 1])$ qaytarmalıdır. Məsələn, $X = [2, 1, 0, 1, 2, 3]$ və $Y = [-2, 2, -1, 0, 0, 1]$ kimi digər keçərli strategiyalar da düzgün hesab olunur.

Bu nümunə 2-ci və 3-cü alt tapşırıqların məhdudiyyətlərini ödəyir.

Nümunə Qiymətləndirici

Girişin ilk sətirində ssenarilərin sayı olan tək bir T tam ədədi olmalıdır. Aşağıda göstərilən formatda T ssenarinin təsviri davam etməlidir.

Giriş formatı:

```
N
A[0] A[1] ... A[N-1]
B[0] B[1] ... B[N-1]
U[0] V[0]
U[1] V[1]
...
U[N-2] V[N-2]
```

Çıxış formatı:

```
k
X[0] X[1] ... X[k]
Y[0] Y[1] ... Y[k]
```